학습데이터처리

Pandas 기초

© KOPO-Dept.
AI Engineering Limited

판다스 입문

- 인덱스 활용

  - - 특정 열을 행 인덱스로 설정
    1) **set_index()** 메소드 사용 (DataFrame의 특정 열을 행 인덱스로 설정) e.g.) 이름 컬럼을 행으로 설정 등
    2) 새로운 객체를 반환

```
특정 열을 행 인덱스로 설정: DataFrame 객체.set_index( [ '열 이름' ] 또는 '열 이름')
```



[그림 1-16] 데이터프레임의 특정 열을 행 인덱스로 설정

- 행 인덱스 재배열
  1) **reindex()** 메소드 사용 (DataFrame의 행 인덱스를 **새로운 배열로 재지정 가능**)
  2) 기존 객체를 변경하지 않고, 새로운 DataFrame 객체를 반환
  3) 기존 데이터프레임에 존재하지 않는 행 인덱스가 새롭게 추가되는 경우, 그 행의 데이터 값은 NaN 값이 입력
  4) NaN 값 대신 fill_value 파라미터를 이용하여 특정 값을 넣을 수 있음 (e.g. **reindex(new_idx, fill_value=0)**)

```
새로운 배열로 행 인덱스를 재지정: DataFrame 객체.reindex( 새로운 인덱스 배열 )
```

	c0	c1	c2	c3	c4
r0	1	4	7	10	13
r1	2	5	8	11	14
r2	3	6	9	12	15

	c0	c1	c2	c3	c4
r0	1.0	4.0	7.0	10.0	13.0
r1	2.0	5.0	8.0	11.0	14.0
r2	3.0	6.0	9.0	12.0	15.0
r3	NaN	NaN	NaN	NaN	NaN
r4	NaN	NaN	NaN	NaN	NaN

	c0	c1	c2	c3	c4
r0	1	4	7	10	13
r1	2	5	8	11	14
r2	3	6	9	12	15
r3	0	0	0	0	0
r4	0	0	0	0	0

판다스 입문

- 인덱스 활용

  - 행 인덱스 초기화

    1) reset_index() 메소드 사용 (정수형 인덱스로 초기화)

    2) 기존 행 인덱스는 열로 이동

```
정수형 위치 인덱스로 초기화: DataFrame 객체.reset_index( )
```

〈실행 결과〉 코드 전부 실행



	c0	c1	c2	c3	c4
r0	1	4	7	10	13
r1	2	5	8	11	14
r2	3	6	9	12	15

	index	c0	c1	c2	c3	c4
0	r0	1	4	7	10	13
1	r1	2	5	8	11	14
2	r2	3	6	9	12	15

  - 행 인덱스를 기준으로 DataFrame 정렬

    1) sort_index() 메소드 사용 (행 인덱스를 기준으로 정렬)

    2) ascending 옵션 사용하여 오름차순 또는 내림차순(ascending=False) 설정

```
행 인덱스 기준 정렬: DataFrame 객체.sort_index( )
```

〈실행 결과〉 코드 전부 실행

	c0	c1	c2	c3	c4
r0	1	4	7	10	13
r1	2	5	8	11	14
r2	3	6	9	12	15

	c0	c1	c2	c3	c4
r2	3	6	9	12	15
r1	2	5	8	11	14
r0	1	4	7	10	13

판다스 입문

- 인덱스 활용

  - 열 인덱스를 행 인덱스로, 행 인덱스 재배열, 행 인덱스 초기화, 행 인덱스를 기준으로 Dataframe 정렬

```
import pandas as pd

exam_data = {'이름' : ['학생1', '학생2', '학생3'],
             '인공지능기초': [90, 80, 70], '자료구조': [98, 89,
95],
             '자바': [85, 95, 100], '기계학습프로그래밍':
[100, 90, 90]}
df = pd.DataFrame(exam_data)

# print(df.columns, df.columns[0])
new_df = df.set_index(['이름']) # 컬럼이름을 index로
사용하기.
# 컬럼의 인덱스로 가지고 오기 위해서는?
# new_df = df.set_index(df.columns[0])
print(new_df)
```


© KOPO-Dept.
AI Engineering Limited

판다스 입문

- 산술연산

  - 판다스 객체의 산술연산은 **내부적으로 3단계 프로세스**를 거친다.
    1) 행/열 인덱스를 기준으로 모든 원소를 정렬한다.
    2) 동일한 위치에 있는 원소끼리 일대일로 대응시킨다.
    3) 일대일 대응이 되는 원소끼리 연산을 처리한다. 이때, 대응되는 원소가 없으면 NaN으로 처리한다.

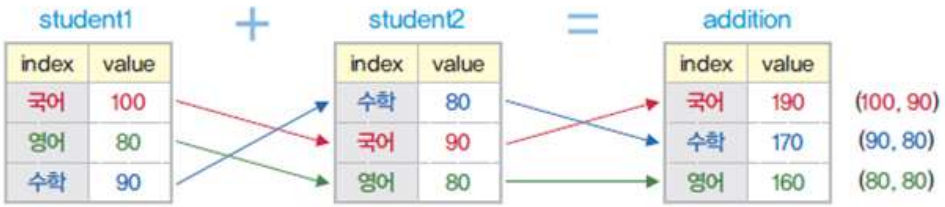
  - **Series** Vs. Number

```
시리즈와 숫자 연산: Series객체 + 연산자(+, -, *, /) + 숫자
```

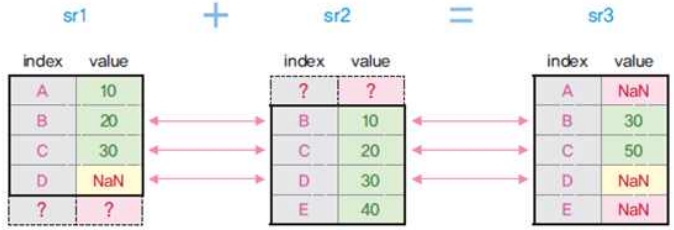
  - **Series** Vs. Series

```
시리즈와 시리즈 연산: Series1 + 연산자(+, -, *, /) + Series2
```

- 객체 사이에 공통 인덱스가 없는 경우에 NaN으로 반환.
- 이런 상황을 피하려면 연산 메소드에 **fill_value 옵션을 설정**



[그림 1-17] 시리즈의 산술연산(덧셈)



[그림 1-18] NaN값이 있는 시리즈의 산술연산(덧셈)

판다스 입문

- 산술연산

  - DataFrame 연산
    1) Series 연산 확장 개념으로,
    2) 행/열 인덱스를 기준으로 정렬하고 동일한 위치에 있는 원소끼리 일대일로 대응 후 연산
    3) 기존 데이터프레임의 형태를 그대로 유지한 채, 원소 값만 새로운 값으로 바뀐다
  - **DataFrame** Vs. Number

```
데이터프레임과 숫자 연산: DataFrame 객체 + 연산자(+, -, *, /) + 숫자
```

  - **DataFrame** Vs. DataFrame

```
데이터프레임의 연산자 활용: DataFrame1 + 연산자(+, -, *, /) + DataFrame2
```

df1

	열 A	열 B	열 C
행 0	15	100	1
행 1	20	500	2
행 2	50	200	3
행 3	NaN	500	2
행 4	NaN	200	3

+

df2

	열 A	열 B
행 0	10	200
행 1	10	100
행 2	10	200
행 3	20	100
행 4	30	100

=

df1+df2

	열 A	열 B	열 C
행 0	25	300	NaN
행 1	30	600	NaN
행 2	60	400	NaN
행 3	NaN	600	NaN
행 4	NaN	300	NaN

[그림 1-19] 데이터프레임의 산술연산(덧셈)

판다스 입문

- 산술 연산

  - Series 연산, DataFrame 연산

```python
import pandas as pd

# 딕셔너리 데이터로 판다스 시리즈 만들기
student1 = pd.Series({'국어':100, '영어':80, '수학':90})
print(student1)
print('\n')

# 학생의 과목별 점수를 100으로 나누기 (0~1 값으로 Normalization)
percentage = student1 / 100

print(percentage)
print('\n')
print(type(percentage))

# Series vs. Series
# 모든 데이터가 다 있는 상태에서 사칙연산 수행
# 사칙연산의 index, 컬럼은 국영수
# 딕셔너리 데이터로 판다스 시리즈 만들기
student1 = pd.Series({'국어':100, '영어':80, '수학':90})
student2 = pd.Series({'수학':80, '국어':90, '영어':80})
```

※ 타이타닉('titanic') 데이터셋

Seaborn 라이브러리에서 제공하는 데이터셋으로, 타이타닉호 탑승자에 대한 인적사항과 구조 여부 등을 정리한 자료이다.

load_dataset() 함수로 불러온다.

[Seaborn 내장 데이터셋의 종류]

'anscombe', 'attention', 'brain_networks', 'car_crashes', 'diamonds', 'dots', 'exercise', 'flights', 'fmri', 'gammas', 'iris', 'mpg', 'planets', 'tips', 'titanic'

판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 목표: 분류 문제의 "**성능과 상관없이 데이터에서 불필요한 데이터를 제거해서 학습**"시켜 보자

- 1) 판단스 기초
  - 1-1. 데이터 살펴보기(데이터분석: 각종 통계 등) 중에서 필요한 것
    - Dataframe.head()
    - Dataframe.info()
  - 1-2. 데이터 전처리(이상치, 결측치 처리, 범주형 카테고리 처리 등) 중에서 필요한 것
    - 범주형 카테고리 처리: pandas.get_dummies
  - 1-3. 데이터 응용 (함수매핑, 필터링, 병합 등) 중에서 필요한 것
    - 필터링 중에서: Boolean Indexing: Dataframe[Boolean Series]
    - 병합: pandas.concat

- 2) 분류기: SVM (Support Vector Machine)

© KOPO-Dept.
AI Engineering Limited

판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 1) 판단스 기초
  - 1.1. 데이터 살펴보기
    - Dataframe.head()
      - **head(n)**: 정수 n을 입력하면 처음 n개의 행을 보여줌 (Default n: 5)

```
pd.set_option('display.max_columns', 20)
titanic_data = sns.load_dataset("titanic")
```

» https://seaborn.pydata.org/generated/seaborn.load_dataset.html

컬럼명	설명	컬럼명	설명
survived	생존 여부 0: 사망, 1: 생존	**class**	객실등급 First면 1등급, Second면 2등급, Third면 3등급
pclass	객실등급	**who**	남자, 여자, 아이 Man, Woman, Child
sex	성별	**adult_male**	성인 남자 여부
age	나이	**deck**	선실 번호 첫 알파벳
sibsp	함께 탑승한 형제 및 배우자 수	**Embarked_town**	탑승지 이름
parch	함께 탑승한 자녀 및 부모 수	**alive**	생존여부 no면 사망, yes면 생존
fare	요금	**alone**	혼자 탑승 여부 True면 혼자 탑승, False면 가족과 함께 탑승
embarked	탑승지 이름 앞글자 C는 Cherbourg, Q는 Queenstown, S는 Southampton		

판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 1) 판단스 기초
  - 1.1. 데이터 살펴보기
    - Dataframe.head()
    - Dataframe.info()
      - 데이터 프레임에 대한 기본 정보 출력: 클래스 유형, 행 인덱스의 구성, 열 이름의 종류와 개수, 메모리 할당량

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 1) 판단스 기초
  - 1-2. 데이터 전처리(범주형 카테고리 처리)
    - 1) 객체 분류: 개, 고양이, 호랑이 → 컴퓨터가 인식 가능한 입력 값으로 변환
    - 2) dog: 1, cat: 2, tiger: 3 과 같이 처리하는 것보다는
    - One-hot Encoding 방식이 적합 (0, 1로 어떤 특성이 있는지 없는지 여부만을 표시)
    - Pandas.get_dummies
    - Example: 객실 등급

```
class_onehot = pd.get_dummies(titanic_data['pclass'])
print(class_onehot.head())
```

	1st	2nd	3rd
0	0	0	1
1	1	0	0
2	0	0	1
3	1	0	0
4	0	0	1

© KOPO-Dept.
AI Engineering Limited

판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 1) 판단스 기초
  - 1-3. 데이터 응용
    - Boolean Indexing: Dataframe[Boolean Series]: 시리즈 또는 데이터프레임의 데이터 중에서 **"특정 조건식을 만족하는 원소만 따로 추출"**하는 개념
      - 데이터프레임의 각 열은 시리즈 객체이므로, 조건식(<,>,==,etc.)을 적용하면 각 원소가 조건을 만족하는지 여부를 참과 거짓 값으로 표시하여 Boolean 시리즈를 만들 수 있음
      - 이 Boolean 시리즈를 데이터프레임에 대입하면 조건을 만족하는 행들만 선택 가능
      - Example: age열에서 값이 10보다 크거나 같고 20보다 작거나 같은 데이터만 추출

```
df = titanic_data[((titanic_data.age >= 10) & (titanic_data.age <= 20))]
print(df.head())
print(len(titanic_data), len(df))
```

```
    survived  pclass     sex   age  sibsp  parch      fare embarked   class  \
9          1       2  female  14.0      1      0   30.0708        C  Second
12         0       3    male  20.0      0      0    8.0500        S   Third
14         0       3  female  14.0      0      0    7.8542        S   Third
22         1       3  female  15.0      0      0    8.0292        Q   Third
27         0       1    male  19.0      3      2  263.0000        S   First

      who  adult_male deck  embark_town alive  alone
9   child       False  NaN    Cherbourg   yes  False
12    man        True  NaN  Southampton    no   True
14  child       False  NaN  Southampton    no   True
22  child       False  NaN   Queenstown   yes   True
27    man        True    C  Southampton    no  False
891 117
```


Daegu Campus of KOREA POLYTECHNIC

판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 1) 판단스 기초
  - 1-3. 데이터 응용
    - Boolean Indexing
    - pandas.concat

데이터프레임 연결: pandas.concat(데이터프레임의 리스트)

  - 옵션
    - » axis(0: row 방향으로 연결, 1: col 방향으로 연결)
  - Example: “one-hot encoding” 으로 만든 pclass 컬럼을 Concatenate 해보자.

```
df_concat = pd.concat([titanic_data, class_onehot], axis=1)
print(df_concat.head())
```

```
     who  adult_male deck  embark_town alive  alone  1st  2nd  3rd
0    man        True  NaN  Southampton    no  False    0    0    1
1  woman       False    C    Cherbourg   yes  False    1    0    0
2  woman       False  NaN  Southampton   yes   True    0    0    1
3  woman       False    C  Southampton   yes  False    1    0    0
4    man        True  NaN  Southampton    no   True    0    0    1
```


© KOPO-Dept.
AI Engineering Limited

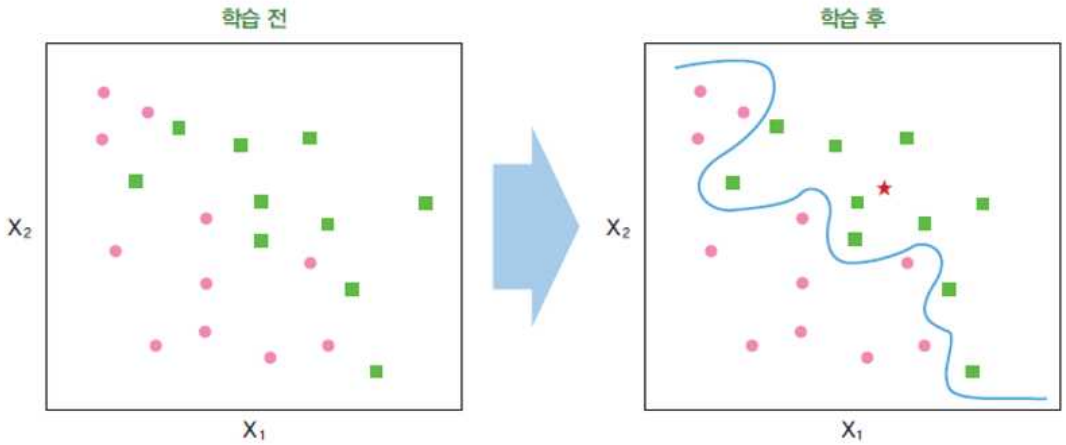
판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 1) 판단스 기초
  - 학습을 위한 데이터 생성
    - 1) 누락된 데이터가 많은 컬럼(e.g. deck) 및 중복 컬럼들(e.g. pclass == class)은 drop
      - 누락 데이터가 별로 없는 컬럼은 값 입력 (최빈도 데이터로 할당)
    - 2) sex, who 컬럼: object → int
    - 3) 범주형 데이터(e.g. embarked)는 one-hot encoding으로 생성
    - 4) concat을 이용하여 학습할 데이터프레임 생성

© KOPO-Dept.
AI Engineering Limited

판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 2) 분류기: SVM (Support **Vector** Machine)
  - Regression, **Classification** 등에 사용되는 지도 학습 방법
    - sklearn.SVM.SVR(Regression), **sklearn.SVM.SVC(Classification)**
  - 데이터 셋의 여러 속성을 나타내는 데이터프레임의 각 열은 **열 벡터 형태로 구현** 가능
  - 속성이 두 개(Feature가 2개)인 경우, 2차원 평면 공간에 좌표로 표시➔ N차원까지 확장 가능
  - **고차원 공간에서 효과적**

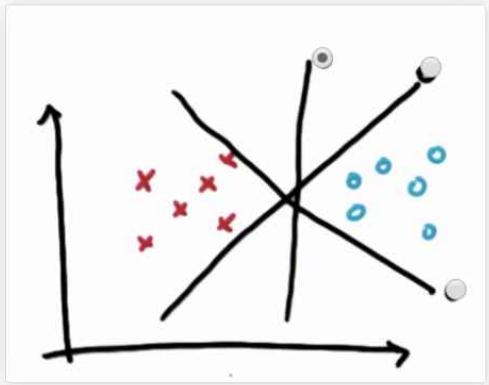


2차원 평면 예시: 학습을 통하여 벡터 공간을 나누는 **margin이 최대인 경계를 찾는 것**

© KOPO-Dept.
AI Engineering Limited

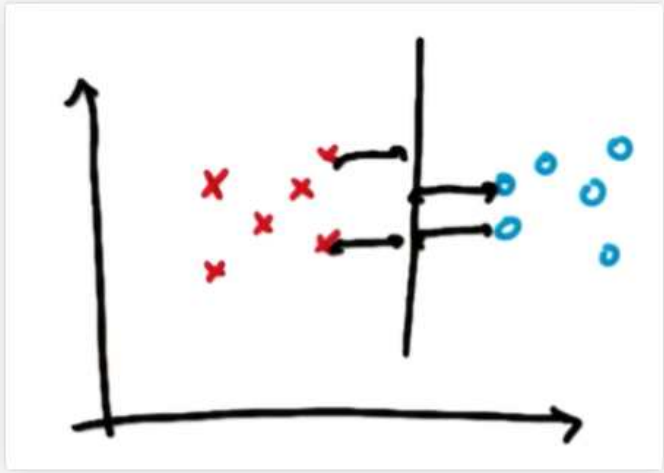
판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 2) 분류기: SVM (Support **Vector** Machine)



출처: Udacity

Margin의 최대화



출처: Udacity

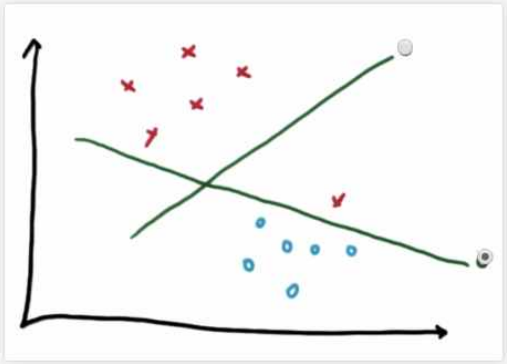
출처: https://bkshin.tistory.com/entry/%EB%A8%B8%EC%8B%A0%EB%9F%AC%EB%8B%9D-2%EC%84%9C%ED%8F%AC%ED%8A%B8-%EB%B2%A1%ED%84%B0-%EB%A8%B8%EC%8B%A0-SVM

판다스 입문: 타이타닉 생존여부 예측 분류 문제

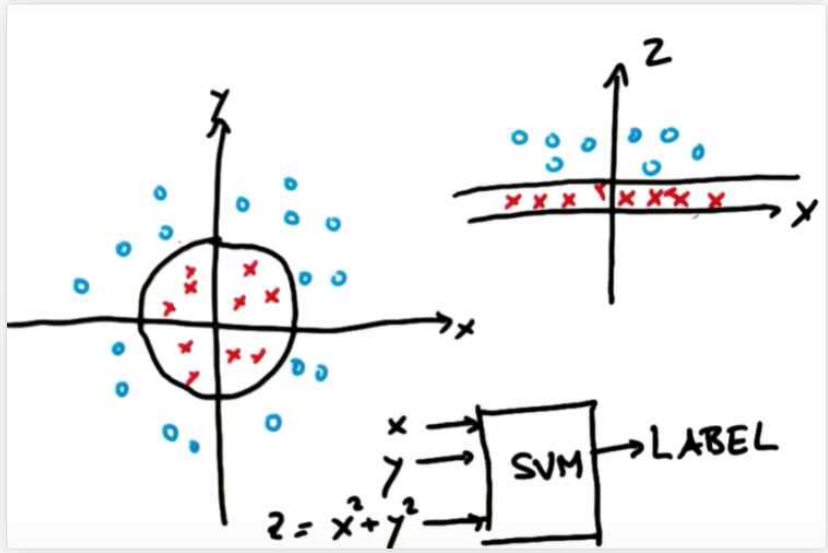
- 2) 분류기: SVM (Support **Vector** Machine)

데이터를 정확히 분류하면서 margin이 큰 경계선을 찾는 방법

데이터의 정확한 분류



출처: Udacity



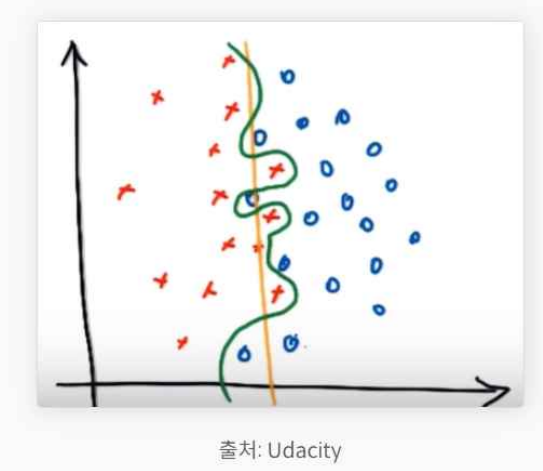
출처: Udacity

고차원 공간에서 효율: 커널 트릭 제공

- 커널트릭: **저차원 공간(low dimensional space)을 고차원 공간(high dimensional space)으로 매핑해주는 작업을 커널 트릭**

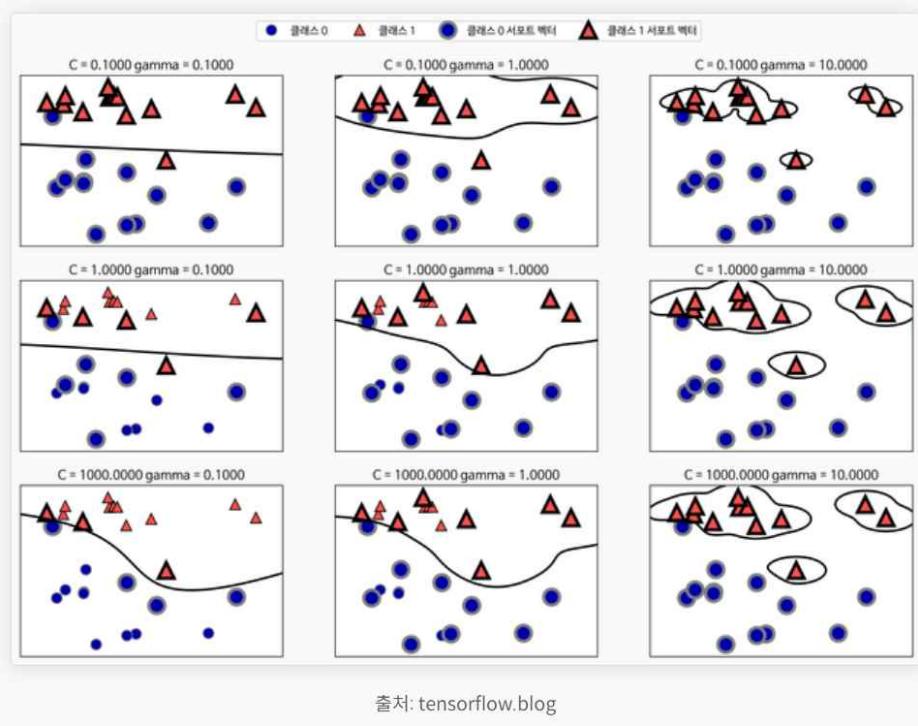
판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 2) 분류기: SVM (Support **Vector** Machine)
  - sklearn svm.svc parameter
    - Kernel: rbf(Radial Basis Function: default)
    - C: **값이 크면 decision boundary는 더 굴곡지고, 작으면 decision boundary는 직선에 가까움**
      - **데이터를 잘 구분하는 것에 초점**
    - Gamma: 동일함
      - **Decision Boundary를 만드는 것에 초점**



출처: Udacity

C parameter



출처: tensorflow.blog

판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 2) 분류기: SVM (Support **Vector** Machine)

```
svm_model = svm.SVC(kernel='rbf')

# train data로 모델 학습
svm_model.fit(x_train, y_train)

# test data를 가지고 y_hat 예측
y_hat = svm_model.predict(x_test)

print(y_hat[0:10])
print(y_test.values[0:10])

# 학습 모델 성능 평가
from sklearn import metrics
svm_matrix = metrics.confusion_matrix(y_test, y_hat)
print(svm_matrix)
```

```
[[160  14]
 [ 48  46]]
              precision    recall  f1-score

           0       0.77      0.92      0.84
           1       0.77      0.49      0.60

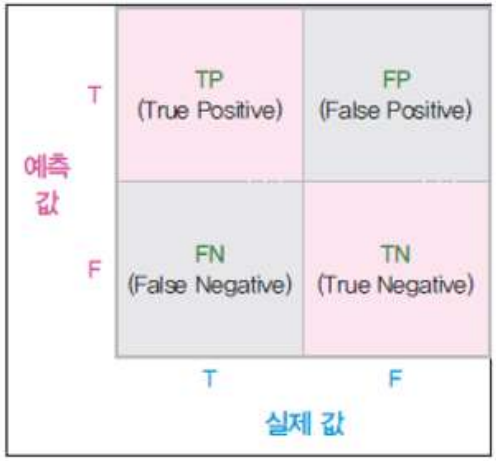
    accuracy                           0.77
   macro avg       0.77      0.70      0.72
weighted avg       0.77      0.77      0.75
```

판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 2) 분류기: SVM (Support **Vector** Machine)
  - 평가지표

❶ Confusion Matrix

모형이 예측하는 값에는 두 가지(True/False)가 있고, 각 예측값은 실제로 True이거나 False일 수 있다. 다음 그림과 같이 모형의 예측값과 실제 값을 각각 축으로 하는 2 X 2매트릭스로 표현한 것을 Confusion Matrix라고 부른다.



판다스 입문: 타이타닉 생존여부 예측 분류 문제

- 2) 분류기: SVM (Support **Vector** Machine)
  - 평가지표

❷ 정확도(Precision)

True로 예측한 분석대상 중에서 실제 값이 True인 비율을 말하며, 모형의 정확성을 나타내는 지표가 된다. 정확도가 높다는 것은 False Positive(실제 False를 True로 잘못 예측) 오류가 작다는 뜻이다.

$$\text{Precision} = \frac{TP}{TP + FP}$$

❸ 재현율(Recall)

실제 값이 True인 분석대상 중에서 True로 예측하여 모형이 적중한 비율을 말하며, 모형의 완전성을 나타내는 지표이다. 재현율이 높다는 것은 False Negative(실제 True를 False로 잘못 예측) 오류가 낮다는 뜻이다.

$$\text{Recall} = \frac{TP}{TP + FN}$$

❹ F1 지표(F1 -score)

정확도와 재현율이 균등하게 반영될 수 있도록 정확도와 재현율의 조화평균을 계산한 값으로, 모형의 예측력을 종합적으로 평가하는 지표이다. 값이 높을수록 분류 모형의 예측력이 좋다고 말할 수 있다.

$$\text{F1 score} = \frac{2 * (\text{Precision} * \text{Recall})}{\text{Precision} + \text{Recall}}$$

학습데이터처리

데이터 입출력

© KOPO-Dept.
AI Engineering Limited

데이터 입출력

- 외부 파일 읽기
  - 판다스 데이터 입출력 도구
    1) 판다스는 다양한 형태의 외부 파일을 읽어와서 데이터프레임으로 변환하는 함수를 제공
    2) 어떤 파일이든 판다스 객체인 데이터프레임으로 변환되고 나면, 판다스의 모든 함수와 기능을 자유롭게 사용
    3) 데이터 프레임을 다양한 유형의 파일로 저장 가능

File Format	Reader	Writer
CSV	read_csv	to_csv
JSON	read_json	to_json
HTML	read_html	to_html
Local clipboard	read_clipboard	to_clipboard
MS Excel	read_excel	to_excel
HDF5 Format	read_hdf	to_hdf
SQL	read_sql	to_sql

[표 2-1] 판다스 데이터 입출력 도구(출처: http://pandas.pydata.org)

데이터 입출력

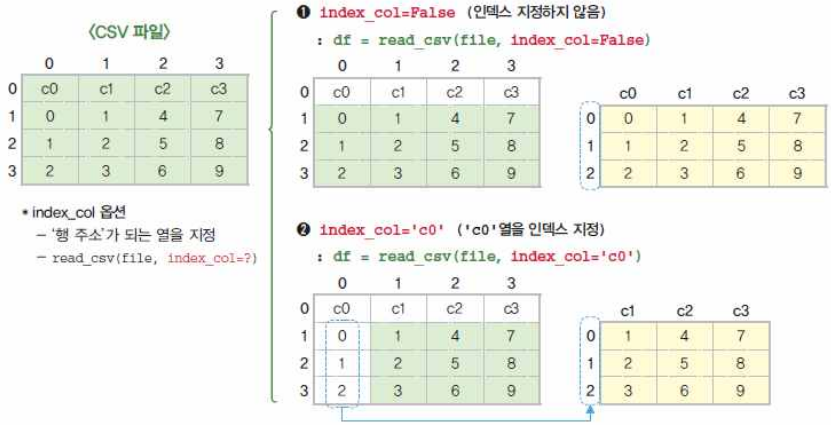
- 외부파일 읽기: CSV 파일

  - 데이터 값을 쉼표(,)로 구분하고 있다는 의미로 CSV(comma-separated values)라고 부르는 텍스트 파일
  - 쉼표(,)로 열을 구분하고 줄바꿈으로 행을 구분
  - read_csv() 함수에 확장자(.csv)를 포함하여 파일경로(파일명)을 입력하면, CSV 파일을 읽어와서 데이터프레임으로 변환

CSV 파일 → 데이터프레임: `pandas.read_csv("파일 경로(이름)")`



[그림 2-1] CSV 파일 읽기 – header 옵션 비교



[그림 2-2] CSV 파일 읽기 – index_col 옵션 비교

데이터 입출력

- 외부 파일 읽기: Excel, Json 파일
  - Excel 파일: **read_excel()** 메소드 사용 (read_csv()와 거의 비슷함, header, index_col 등 대부분 옵션 그대로 사용 가능)

Excel 파일 → 데이터프레임: `pandas.read_excel("파일 경로(이름)")`

  - JSON(JavaScript Object Notation) 파일: 구조화된 데이터를 표현하기 위한 문자 기반의 표준 포맷
    - 파이썬 딕셔너리와 비슷하게 'key : value' 구조
    - **read_json()** 함수를 사용하여, JSON 파일을 데이터프레임으로 변환

JSON 파일 → 데이터프레임: `pandas.read_json("파일 경로(이름)")`

```
{
    "name":{"pandas":"",
            "NumPy":"",
            "matplotlib":""},

    "year":{"pandas":2008,
            "NumPy":2006,
            "matplotlib":2003},

    "developer":{"pandas":"Wes Mckinneye",
                 "NumPy":"Travis Oliphant",
                 "matplotlib":"John D. Hunter"},

    "opensource":{"pandas":"True",
                  "NumPy":"True",
                  "matplotlib":"True"}
}
```

〈실행 결과〉 코드 전부 실행

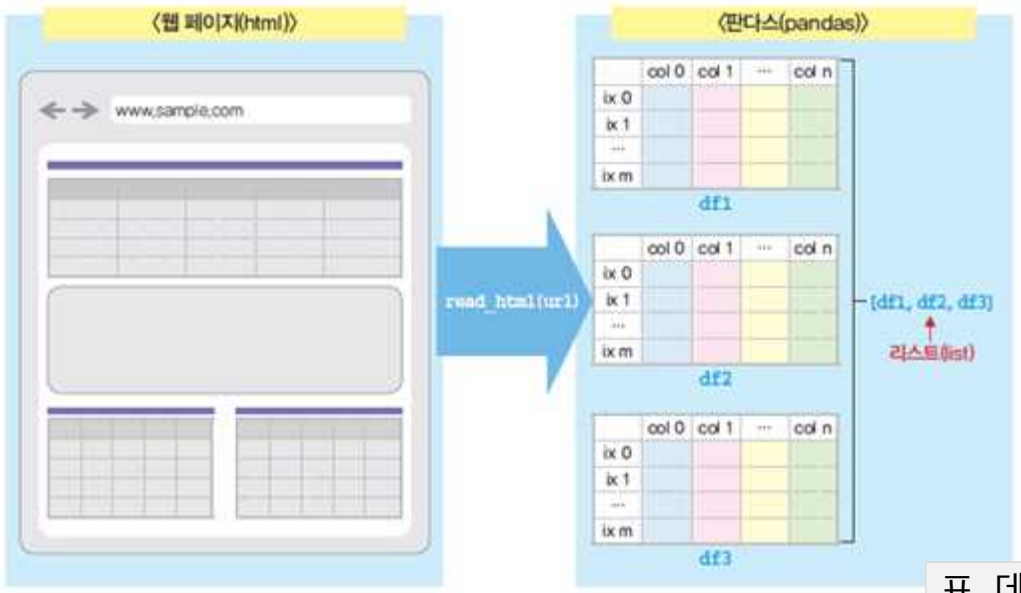
```
name          year          developer    opensource
NumPy         2006    Travis Oliphant          True
matplotlib    2003     John D. Hunter          True
pandas        2008      Wes Mckinneye          True


Index(['NumPy', 'matplotlib', 'pandas'], dtype='object')
```

데이터 입출력

- 외부 파일 읽기: HTML 웹 페이지에서 표 속성 가져오기
  - **read_html()** 함수는 HTML 웹 페이지에 있는 <table> 태그에서 표 형식의 데이터를 모두 찾아서 데이터프레임으로 변환

HTML 표 속성 읽기 : `pandas.read_html( "웹 주소(URL)" 또는 "HTML 파일 경로(이름)" )`



[그림 2-3] HTML 페이지의 표 가져오기

표 데이터들은 각각 별도의 데이터프레임으로 변환되기 때문에, 여러 개의 데이터프레임(표)을 원소로 갖는 리스트가 반환된다.

© KOPO-Dept.
AI Engineering Limited

데이터 입출력

- 외부파일 읽기: CSV, Excel, JSON 파일

```python
import pandas as pd

# 파일경로를 찾고, 변수 file_path에 저장
file_path = './read_csv_sample.csv'

# read_csv() 함수로 데이터프레임 변환. 변수 df1에 저장
df1 = pd.read_csv(file_path)
print(df1)
print('\n')

# read_csv() 함수로 데이터프레임 변환. 변수 df2에 저장. header=None
df2 = pd.read_csv(file_path, header=None)
print(df2)
print('\n')

# read_csv() 함수로 데이터프레임 변환. 변수 df3에 저장. index_col=No
df3 = pd.read_csv(file_path, index_col=None)
print(df3)
print('\n')
```

옵션	설명
path	파일의 위치(파일명 포함), URL
sep(또는 delimiter)	텍스트 데이터를 필드별로 구분하는 문자
header	열 이름으로 사용될 행의 번호(기본값은 0) header가 없고 첫 행부터 데이터가 있는 경우 None으로 지정 가능
index_col	행 인덱스로 사용할 열의 번호 또는 열 이름
names	열 이름으로 사용할 문자열의 리스트
skiprows	처음 몇 줄을 skip할 것인지 설정(숫자 입력) skip하려는 행의 번호를 담은 리스트로 설정 가능(예: [1, 3, 5])
parse_dates	날짜 텍스트를 datetime64로 변환할 것인지 설정(기본값은 False)
skip_footer	마지막 몇 줄을 skip할 것인지 설정(숫자 입력)
encoding	텍스트 인코딩 종류를 지정(예: 'utf-8')

[표 2-2] read_csv( ) 함수의 옵션

© KOPO-Dept.
AI Engineering Limited

데이터 입출력

- 외부파일 읽기: HTML 파일
- CSV 파일로 출력

```python
import pandas as pd

# HTML 파일 경로 or 웹 페이지 주소를 url 변수에 저장
url ='./sample.html'

# HTML 웹페이지의 표(table)를 가져와서 데이터프레임으로 변환
tables = pd.read_html(url)

# 표(table)의 개수 확인
print(len(tables))
print('\n')

# tables 리스트의 원소를 iteration하면서 각각 화면 출력
for i in range(len(tables)):
    print("tables[%s]" % i)
    print(tables[i])
    print('\n')

# 파이썬 패키지 정보가 들어 있는 두 번째 데이터프레임을 선택하여
df = tables[1]
```

Writer
to_csv
to_json
to_html
to_clipboard
to_excel
to_hdf
to_sql

//pandas.pydata.org

```python
import pandas as pd

# 판다스 DataFrame() 함수로 데이터프레임 변환. 변수 df에 저장
data = {'name': ['Jerry', 'Riah', 'Paul'],
        'algol': ["A", "A+", "B"],
        'basic': ["C", "B", "B+"],
        'c++': ["B+", "C", "C+"],
        }

df = pd.DataFrame(data)
df.set_index('name', inplace=True)   #name 열을 인덱스로 지정
print(df)

# to_csv() 메소드를 사용하여 CSV 파일로 내보내기. 파열명은 df_sample.csv로 저장
df.to_csv("./df_sample.csv")
```


© KOPO-Dept.
AI Engineering Limited

데이터 입출력

- Excel 파일로 출력 (여러 개의 데이터 프레임을 하나의 Exel 파일로 저장)

데이터프레임 여러 개를 Excel 파일로 저장: `pandas.ExcelWriter( "파일 이름(경로)" )`

```python
import pandas as pd

# 판다스 DataFrame() 함수로 데이터프레임 변환. 변수 df1, df2에 저장
data1 = {'name' : ['Jerry', 'Riah', 'Paul'],
         'algol' : ["A", "A+", "B"],
         'basic' : ["C", "B", "B+"],
          'c++' : ["B+", "C", "C+"]}

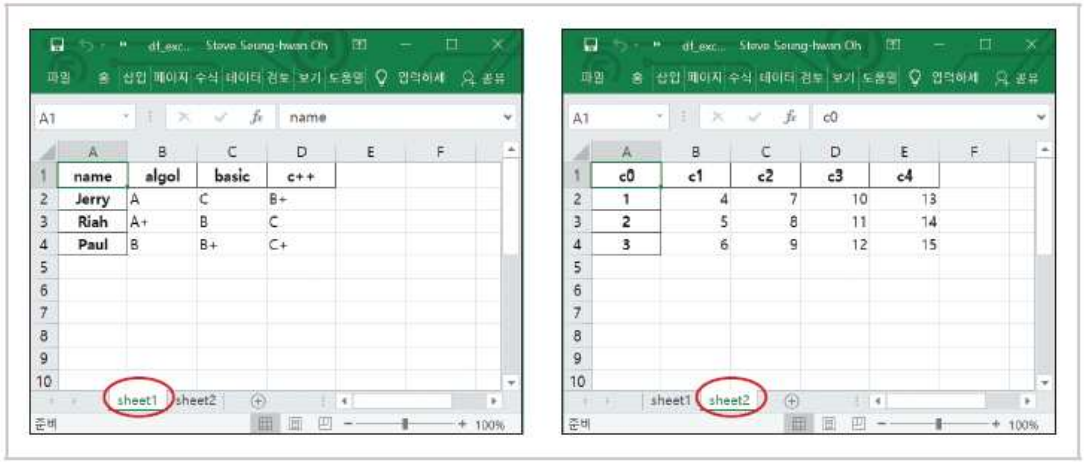
data2 = {'c0':[1,2,3],
         'c1':[4,5,6],
         'c2':[7,8,9],
         'c3':[10,11,12],
         'c4':[13,14,15]}

df1 = pd.DataFrame(data1)
df1.set_index('name', inplace=True)      #name 열을 인덱스로 지정
print(df1)
print('\n')

df2 = pd.DataFrame(data2)
df2.set_index('c0', inplace=True)        #c0 열을 인덱스로 지정
```

판다스 ExcelWriter() 함수는 Excel 워크북 객체를 생성

〈실행 결과〉 코드 전부 실행 ② – Excel 파일 내용 보기 (File: example/part2/df_excelwriter.xlsx)



Thank you

Q&A

www.kopo.ac.kr
jsshin7@kopo.ac.kr

 © KOPO-Dept.
AI Engineering Limited